



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI

Imię i nazwisko studenta: Jakub Ławicki

Nr albumu: 189788

Poziom kształcenia: studia drugiego stopnia

Forma studiów: stacjonarne

Kierunek studiów: Elektronika i telekomunikacja

Specjalność: Systemy wbudowane czasu rzeczywistego

PRACA DYPLOMOWA MAGISTERSKA

Tytuł pracy w języku polskim: Badanie wpływu różnych metod optymalizacji kodu na wydajność systemu wbudowanego opartego na systemie Linux.

Tytuł pracy w języku angielskim: Investigation of the impact of various code optimization methods on the performance of a Linux-based embedded system.

Opiekun pracy: dr hab. inż. Jacek Marszał

STRESZCZENIE

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec odio sapien, egestas a ullamcorper a, volutpat tempor felis. Proin vel laoreet augue, a tincidunt ex. Nam maximus massa nibh, ac convallis nunc viverra ac. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Maecenas felis mi, venenatis id iaculis eget, aliquam sit amet purus. Ut vehicula convallis finibus. Nam vulputate commodo magna, eu dapibus odio laoreet in. Phasellus tempus bibendum elementum. Pellentesque dapibus pulvinar orci nec sodales. Etiam eu augue non ex sodales sollicitudin vitae quis mi. Nulla vitae elit ac nisl egestas tempor. Nam ut nisl gravida, convallis purus sed, dapibus augue.

Słowa kluczowe: Optymalizacja algorytmów, systemy wbudowane, system operacyjny Linux

Dziedzina nauki i techniki zgodna z OECD Nauki inżynieryjne i techniczne, Elektrotechnika, elektronika i inżynieria informatyczna,

ABSTRACT

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec odio sapien, egestas a ullamcorper a, volutpat tempor felis. Proin vel laoreet augue, a tincidunt ex. Nam maximus massa nibh, ac convallis nunc viverra ac. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Maecenas felis mi, venenatis id iaculis eget, aliquam sit amet purus. Ut vehicula convallis finibus. Nam vulputate commodo magna, eu dapibus odio laoreet in. Phasellus tempus bibendum elementum. Pellentesque dapibus pulvinar orci nec sodales. Etiam eu augue non ex sodales sollicitudin vitae quis mi. Nulla vitae elit ac nisl egestas tempor. Nam ut nisl gravida, convallis purus sed, dapibus augue.

Keywords: Self-organizing maps, unsupervised learning, clustering, classification, WTM algorithm

OECD consistent field of science and technology classification: Engineering and technology, Electrical engineering, Electronic engineering, Information engineering,

SPIS TREŚCI

Spis treści	5
1 Wstęp i cel pracy	8
1.1 Cel pracy	8
1.2 Zawartość rozdziałów	8
2 Architektura systemów operacyjnych w zastosowaniach wbudowanych	9
2.1 Rola systemów operacyjnych w systemach wbudowanych	9
2.1.1 Model warstwowy systemów operacyjnych	10
2.2 Kluczowe abstrakcje i mechanizmy stosowane w systemach operacyjnych	11
2.2.1 Tryby pracy procesora	12
2.2.2 Procesy	12
2.2.3 Wątki	12
2.2.4 Szeregowanie zadań	13
2.2.5 Pamięć wirtualna	13
2.2.6 Mechanizmy synchronizacji i współbieżności	13
2.3 Systemy wbudowane czasu rzeczywistego wraz z ich podziałem	13
2.4 Architektura struktur jądra systemów operacyjnych	15
2.5 Studium przypadku systemu operacyjnego Linux	16
2.5.1 Procesy, wątki w systemie Linux	17
2.5.2 Szeregowanie i synchronizacja w systemie Linux	17
2.5.3 Zarządzanie pamięcią	18
3 Wybrany algorytm przetwarzania danych i techniki optymalizacji programów komputerowych	19
3.1 Opis i analiza Szybkiej Transformaty Fouriera	19
3.2 Warianty algorytmu Szybkiej Transformaty Fouriera	19
3.3 Opis procesu kompilacji programu komputerowego	19
3.4 Przegląd najpopularniejszych kompilatorów języka C	19
3.5 Poziomy optymalizacji dostępne w kompilatorach Clang i Gcc	19
3.6 Zaawansowane techniki optymalizacji kodu	19
4 Projekt systemu testowego i metodyka przeprowadzonych badań	20
4.1 Założenia projektowe i architektura aplikacji	20
4.2 Opis platformy sprzętowej	20
4.3 Użyte narzędzia programistyczne	20
4.4 Metodyka przeprowadzonych pomiarów	20
4.5 Prezentacja przygotowywanych wariantów programu	20
4.6 Implementacja wersji wielowątkowej, opis biblioteki pthread	20
4.7 Weryfikacja poprawności obliczeń z biblioteka zewnętrzna	20

5 Analiza wyników przeprowadzonych badań	21
5.1 Analiza wpływu flag kompilatora na kod sekwencyjny	21
5.2 Analiza wpływu optymalizacji ręcznej programu sekwencyjnego	21
5.3 Analiza skalowalności rozwiązania wielowątkowego	21
5.4 Zestawienie zbiorcze i dyskusja wyników	21
6 Podsumowanie i wnioski	22
6.1 Wnioski końcowe	22
6.2 kierunek dalszych badań	22
Spis rysunków	23
Spis tabel	24

LISTA SYMBOLI

LISTA SKRÓTÓW

1. WSTĘP I CEL PRACY

1.1. Cel pracy

1.2. Zawartość rozdziałów

Ogólną strukturę niniejszej pracy można przedstawić w następujący sposób:

- **Rozdział 2: Charakterystyka systemów wbudowanych opartych na systemie Linux.**
Obejmuje charakterystykę systemów wbudowanych opartych na systemie Linux, z opisem podstawowych pojęć jak jądro systemu, menadżer procesów, zarządzanie pamięcią oraz mechanizmy wielozadaniowości. Opisuje specyfikę zarządzania zasobami sprzętowymi przez jądro systemu. Szczególną uwagę poświęcono mechanizmom wielozadaniowości, szeregowania wątków oraz zarządzania pamięcią wirtualną, które mają kluczowy wpływ na wydajność aplikacji.
- **Rozdział 3: Wybrany algorytm przetwarzania danych i techniki optymalizacji kodu.**
Przedstawia teoretyczne podstawy algorytmu Szybkiej Transformaty Fouriera (FFT) oraz analizę jego złożoności obliczeniowej. Prezentuje różne implementacje tego algorytmu. Omawia proces kompilacji kodu w języku C, nakreślając istotę wyboru kompilatora. Ponadto przedstawia poziomy optymalizacji oferowane przez kompilator GCC, używany w pracy. Wyjaśnia zaawansowane techniki programistyczne, takie jak instrukcje wektorowe oraz optymalizacja pod kątem dostępu do pamięci podręcznej.
- **Rozdział 4: Projekt i implementacja aplikacji testowej.**
Opisuje szczegółowo opracowane oprogramowanie realizujące algorytm Szybkiej Transformaty Fouriera (FFT). Przedstawia alternatywne warianty implementacji kodu: wersję sekwencyjną, wersję zoptymalizowaną pod kątem dostępu do pamięci oraz wersję wielowątkową wykorzystującą mechanizmy współbieżności.
- **Rozdział 5: Środowisko badawcze i metodyka pomiarów.**
Charakteryzuje wykorzystaną platformę sprzętową oraz konfigurację systemu operacyjnego Linux. Opisuje narzędzia programistyczne, biblioteki oraz frameworki użyte do implementacji i optymalizacji kodu. Definiuje scenariusze testowe, przygotowane konfiguracje procesu kompilacji (zestawy flag optymalizacyjnych) oraz narzędzia i metody użyte do pomiaru czasu wykonania i zużycia zasobów systemowych.
- **Rozdział 6: Analiza wpływu optymalizacji na wydajność systemu.**
Prezentuje wyniki przeprowadzonych badań. Każdy podrozdział zawiera zestawienie i analizę wpływu konkretnych czynników – poziomu optymalizacji kompilatora, zmian w kodzie źródłowym oraz liczby wątków – na ostateczną wydajność aplikacji w systemie wbudowanym.
- **Rozdział 7: Podsumowanie i wnioski.**
Stanowi syntezę uzyskanych wyników oraz ocenę skuteczności badanych metod optymalizacji. Zawiera wnioski końcowe dotyczące relacji między nakładem pracy programisty a automatyczną optymalizacją kompilatora oraz propozycje dalszych kierunków badań.

2. ARCHITEKTURA SYSTEMÓW OPERACYJNYCH W ZASTOSOWANIACH WBUDOWANYCH

Projektowanie złożonych systemów komputerowych, zwłaszcza takich jak rozwiązania bazujące na implementacji algorytmów przetwarzania sygnałów w rygorze czasu rzeczywistego przy jednocześnie ograniczonych zasobach sprzętowych, jest zadaniem, które wymaga wielu przemyślanych decyzji przy doborze architektury tworzonego systemu.

Wybór odpowiedniej platformy programowej jest jedną z kluczowych decyzji projektowych. Projektant systemu wbudowanego staje wówczas często przed wyborem między podejściem bezsystemowym, zapewniającym pełną kontrolę nad sprzętem, a zastosowaniem systemu operacyjnego, który wprowadza warstwę abstrakcji kosztem częściowej utraty determinizmu czasowego oraz zwiększenia zużycia fizycznych zasobów systemu [1]. Decyzja ta ma bezpośredni wpływ na poprawność systemu, nie tylko z uwagi na osiąganą wydajność obliczeniową, ale również poprzez bezpieczeństwo i stabilność systemu. W celu zwiększenia jakości projektowanego systemu inżynier oprogramowania powinien posiadać niezbędną wiedzę teoretyczną z zakresu działania systemów operacyjnych, aby zagwarantować tworzenie możliwie wydajnych i niezawodnych systemów komputerowych.

Celem prezentowanego rozdziału jest usystematyzowanie wiedzy niezbędnej do zrozumienia, w jaki sposób architektura stosowanego systemu operacyjnego wpływa na wydajność uruchamianych na nim programów. Rozważania rozpoczynają się od przedstawienia warstwowego modelu systemów operacyjnych oraz kluczowych mechanizmów jądra, takich jak tryby pracy procesora, procesy, wątki, szeregowanie zadań, pamięć wirtualna oraz synchronizacja współbieżnych operacji. Na tej podstawie omówiony zostaje podział systemów wbudowanych czasu rzeczywistego, a także dwa podstawowe modele budowy jądra: architektura monolityczna oraz mikrojądrowa. Ostatnia część rozdziału zawiera studium przypadku systemu operacyjnego Linux, w którym w skrócony sposób starano się przedstawić niezbędne szczegóły implementacji mechanizmów systemowych.

2.1. Rola systemów operacyjnych w systemach wbudowanych

Podczas projektowania systemu wbudowanego, włączając w to system wbudowany czasu rzeczywistego, projektant staje przed wyborem jednego z dwóch podstawowych podejść do organizacji i kontroli całego środowiska. Pierwszym z nich jest **podejście bezsystemowe** (ang. *bare-metal*), w którym całe oprogramowanie funkcjonuje bez żadnej warstwy pośredniej, mając bezpośredni i wyłączny dostęp do fizycznych układów procesora oraz peryferiów. Choć pozwala to na pełną kontrolę nad sprzętem i minimalne narzuty czasowe, to drastycznie ogranicza skalowalność każda zmiana komponentu lub rozbudowa funkcji wymusza wówczas głęboką modyfikację struktury całego kodu.

Alternatywą jest zastosowanie **systemu operacyjnego**, który możemy zdefiniować jako podstawowe oprogramowanie pośredniczące między sprzętem komputerowym a użytkownikiem, którego głównym zadaniem jest zarządzanie i przydzielanie zasobów sprzętowych oraz tworzenie środowiska pozwalającego na wykonywanie programów zaimplementowanych przez jego użytkownika [2].

Jak wynika z przytoczonej definicji, stosowanie systemu operacyjnego pomaga rozwiązy-

wać niektóre problemy napotymane przy stosowaniu podejścia bezsystemowego. Dzieje się tak poprzez całościowe odizolowanie warstwy programowej od fizycznych struktur sprzętu. System operacyjny działa jako nadrzędny pośrednik, przejmując na siebie zadanie nadzorowania środowiska wykonawczego i separując logikę działania od konkretnej platformy sprzętowej.

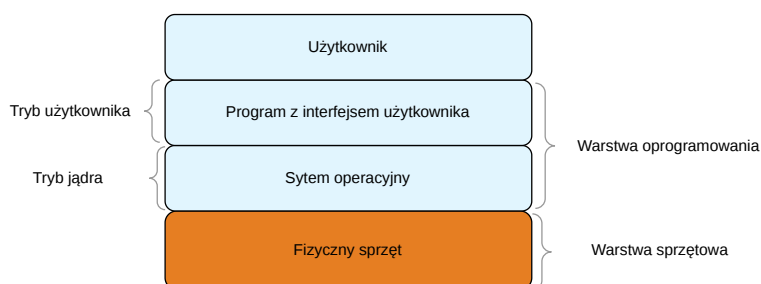
Aby w pełni zrozumieć, jak system operacyjny realizuje te zadania w środowisku wbudowanym, konieczne jest przyjrzenie się jego wewnętrznej architekturze. W dalszych sekcjach zostaną szczegółowo omówione kluczowe mechanizmy oraz podstawowa struktura ogólnych systemów operacyjnych.

2.1.1. Model warstwowy systemów operacyjnych

W inżynierii oprogramowania istotną rolę odgrywa koncepcja **abstrakcji**, polegająca na celowym ukryciu szczegółów implementacji danej funkcjonalności oprogramowania, przed jej potencjalnym użytkownikiem [1]. Podejście to opiera się na założeniu, że należy dostarczyć gotowe funkcjonalność wraz z odpowiednią dokumentacją opisującą sposób jej wykorzystania, zamiast wymagać od użytkownika zrozumienia wewnętrznych mechanizmów działania wytworzonego modułu.

Taka praktyka w sposób znaczący podnosi wygodę wytwarzania nowych elementów systemu, umożliwia zrównoleglenie wytwarzania oprogramowania oraz pozwala na większą specjalizację jej producentów. W celu realizacji komunikacji między kolejnymi modułami systemu stosuje się takie narzędzia jak **interfejsy programistyczne** (ang. *Application Programming Interface*, skrót **API**), czyli zbiór ściśle określonych reguł i sposobów komunikacji, które umożliwiają komunikację między różnymi warstwami oprogramowania. Narzędzie to jest niezbędne, aby w sposób przewidywalny i bezpieczny przekazywać dane lub sterowanie, między różnymi elementami systemu.

Pojęcie abstrakcji jest kluczowe dla zrozumienia architektury współczesnych systemów operacyjnych. W literaturze spotyka się różne podejścia do klasyfikacji poziomów systemu komputerowego [1, 2], jednak ich cechą wspólną jest wyraźne odseparowanie zasobów fizycznych od uruchamianego na nich oprogramowania. Na potrzeby niniejszej pracy przyjęto strukturę opartą na propozycji przedstawionej w [2], którą w graficzny sposób zaprezentowano na rysunku 2.1.



Rysunek 2.1: Warstwowy model systemu komputerowego

Jak pokazano na schemacie, najniższy poziom stanowi warstwa sprzętowa, do której zalicza się wszystkie fizyczne komponenty urządzenia, takie jak **jednostka centralna** (ang. *Central Processing Unit*, CPU), **pamięć** oraz **urządzenia wejścia-wyjścia** (ang. *Input/Output Devices*, I/O). Ponad nią znajduje się warstwa oprogramowania, na którą składa się zarówno system operacyjny, jak i programy użytkownika. Choć oba te elementy należą do tej samej warstwy, funkcjonują one na innych zasadach. Odróżnia je praca w różnych trybach procesora, różniących

się poziomem uprawnień oraz zakresem dostępu do fizycznych urządzeń. Tryby te nazywają się odpowiednio **trybem jądra** (ang. *kernel mode*), dla systemu operacyjnego, oraz **trybem użytkownika** (ang. *user mode*), dla aplikacji użytkownika. Szczegółowa omówienie przytoczonych trybów oraz mechanizmów ich przełączania zostanie omówiona w dalszych częściach tego rozdziału.

Warto przy tym podkreślić, że w przytoczonej pozycji literatury [2] jako integralny komponent struktury systemu komputerowego uwzględnia się jego użytkownika. Zabieg ten ma na celu jednoznaczne wskazanie, że przy projektowaniu dowolnego systemu komputerowego należy zawsze pamiętać o jego ostatecznym przeznaczeniu oraz o użytkowniku końcowym.

W modelu warstwowym architektury komputerowej system operacyjny pełni szczególną rolę, zarządzając granicą między aplikacjami a warstwą fizyczną. W celu zapewnienia poprawnego i przewidywalnego działania systemu wprowadza poziomy abstrakcji, które są niezbędne z kilku istotnych powodów:

1. **Konieczność zapewnienia bezpieczeństwa** - Współczesne systemy operacyjne charakteryzują się ogromną złożonością, a ich kod źródłowy sięga kilkudziesięciu milionów linii [1]. Izolacja poszczególnych poziomów wprowadza wyraźną hierarchię uprawnień. Dzięki temu błąd powstały w kodzie aplikacyjnym pozostaje odizolowany i nie wpływa bezpośrednio na stabilność pozostałych funkcjonalności systemowych ani na pracę samego sprzętu.
2. **Zapewnienie przenoszalności oprogramowania** - Zastosowanie warstw pośrednich sprawia, że oprogramowanie nie jest bezpośrednio uzależnione od konkretnych urządzeń fizycznych. Pozwala to na uruchamianie tych samych programów na różnorodnych platformach sprzętowych, umożliwiając przy tym niezależny rozwój warstwy aplikacyjnej i sprzętowej.
3. **Umożliwienie korzystania z narzędzi wysokiego poziomu** - Ukrycie technicznych szczegółów pracy ze sterownikami lub rejestrarami pozwala na tworzenie kodu w językach wysokiego poziomu. Programista może skupić się na logice i strukturze zgodnej ze sposobem myślenia człowieka, zamiast na bezpośredniej, niskopoziomowej obsłudze układów elektronicznych.

2.2. Kluczowe abstrakcje i mechanizmy stosowane w systemach operacyjnych

Pomimo dużej różnorodności współczesnych systemów operacyjnych, ich architektura opiera się na spójnym zestawie uniwersalnych abstrakcji i mechanizmów. Od strony koncepcyjnej rozwiązania te wprowadzają ład w złożonym środowisku sprzętowym, przybliżając sposób funkcjonowania komputera do logiki myślenia i pracy człowieka.

Koncepcje opisane w prezentowanej sekcji pozwalają na stworzenie przejrzystego i uporządkowanego modelu zarządzania zasobami w systemie operacyjnym. Zrozumienie tych bazowych koncepcji jest kluczowe dla analizy wydajności, ponieważ to one determinują sposób, w jaki program aplikacji, stworzonej przez użytkownika, jest uruchamiany i wykonywany na fizycznym sprzęcie.

Na potrzeby niniejszej pracy konieczne jest przedstawienie następujących pojęć i mechanizmów:

- Tryby pracy procesora
- Proces
- Wątek

- Szeregowanie zadań
- Pamięć wirtualna
- Mechanizmy synchronizacji i współbieżności

2.2.1. Tryby pracy procesora

Współczesne komputery realizują ochronę zasobów poprzez tryby pracy procesora [2], z których kluczowe to **tryb użytkownika** (ang. user mode) oraz **tryb jądra** (ang. kernel mode). Aplikacje użytkownika funkcjonują w trybie użytkownika, nieuprzywilejowanym, co uniemożliwia im między innymi, bezpośredni dostęp do urządzeń peryferyjnych oraz fizycznej pamięci.

Wszelkie operacje wymagające kontroli nad sprzętem realizowane są za pośrednictwem **wywołań systemowych** [2] (ang. system calls), które przełączają procesor w uprzywilejowany tryb jądra.

2.2.2. Procesy

Proces (ang. process) można rozumieć jako wykonujący się program komputerowy [2]. Jego kluczową cechą jest przydzielona, odizolowana od innych procesów wirtualna **przestrzeń adresowa**. Jej wprowadzenie ma zagwarantować bezpieczeństwo – błąd w obrębie jednego procesu nie wpływa bezpośrednio na stabilność procesów systemowych i innych procesów użytkownika.

Współczesne systemy komputerowe składają się z licznych komponentów i aplikacji, co sprawia, że na wspólnych, fizycznych zasobach sprzętowych funkcjonuje jednocześnie wiele procesów. Ze względu na to, że pojedynczy rdzeń procesora może w danej chwili realizować tylko jedno zadanie, system operacyjny stosuje przełączanie kontekstu, cyklicznie przydzielając czas procesora poszczególnym procesom. Ponieważ każdy proces operuje w odrębnej przestrzeni adresowej, zmiana ta wymaga przeładowania pamięci operacyjnej. W efekcie przełączanie kontekstu między procesami jest operacją stosunkowo czasochłonną i wiąże się z zauważalnym opóźnieniem, które można nazywać też **narzutem systemu operacyjnego** [1].

2.2.3. Wątki

Wątek (ang. thread) stanowi wyodrębnioną jednostkę wykonawczą (ciąg instrukcji) uruchamianą na procesorze, funkcjonującą w obrębie konkretnego procesu [1].

W ramach jednego procesu może działać wiele wątków wykonujących się we wspólnej przestrzeni adresowej. Wątki należące do tego samego procesu współdzielą ze sobą zasoby systemowe (m.in. przestrzeń adresową, zmienne globalne czy otwarte pliki), jednak każdy z nich posiada własny, prywatny kontekst wykonania [1], na który składają się:

- **Licznik instrukcji** (ang. Program Counter),
- **Zestaw rejestrów procesora**,
- **Prywatny stos** (przechowujący zmienne lokalne oraz historię wywołań funkcji),
- **Stan wątku** (np. uruchomiony, zablokowany, gotowy).

Dzięki współdzieleniu przestrzeni adresowej wątki generują znacznie mniejszy narzut czasowy niż procesy, przy przełączaniu, wymagając jednak stosowania odpowiednich mechanizmów synchronizacji.

Zarządzanie wątkami może być realizowane w przestrzeni użytkownika lub bezpośrednio w jądrze systemu. W celu zapewnienia przenoszalności oprogramowania wielowątkowego między różnymi środowiskami operacyjnymi stworzony został standard **POSIX Threads** (znany jako **Pthreads**, zdefiniowany w normie IEEE 1003.1c) [1]. Definiuje on uniwersalne interfejsy programistyczne przeznaczone m.in. do tworzenia, kończenia oraz synchronizacji pracy wątków.

2.2.4. Szeregowanie zadań

Szeregowanie zadań odpowiada za zarządzanie przydziałem czasu procesora. Ponieważ pojedynczy rdzeń CPU nie może wykonywać wielu zadań jednocześnie, **planista** (ang. *scheduler*) decyduje o kolejności i czasie uruchamiania wątków i procesów na podstawie określonych algorytmów [2]. Mechanizm ten zapewnia sprawiedliwy dostęp do zasobów i płynność działania systemu, jednak częsta zmiana wykonywanych zadań generuje bezpośredni narzut czasowy związany z przełączaniem kontekstu.

2.2.5. Pamięć wirtualna

Pamięć wirtualna sprawia, że procesy nie operują bezpośrednio na fizycznej pamięci operacyjnej, lecz na wirtualnej przestrzeni adresowej przydzielanej i mapowanej na sprzęt przez **jednostkę zarządzania pamięcią (MMU (ang. Memory Management Unit))**[1].

W kontekście pamięci wirtualnej kluczowym mechanizmem jest **stronicowanie** [1]. Polega ono na podziale wirtualnej przestrzeni adresowej programu na bloki o stałym rozmiarze, zwane stronami (ang. *pages*), które są dynamicznie odwzorowywane na fizyczną pamięć operacyjną. Operacje tę wykonuje jednostka MMU. Mechanizm zarządzania pamięcią wirtualną zostało opisane, na przykładzie systemu Linux w sekcji 2.5.3

2.2.6. Mechanizmy synchronizacji i współbieżności

2.3. Systemy wbudowane czasu rzeczywistego wraz z ich podziałem

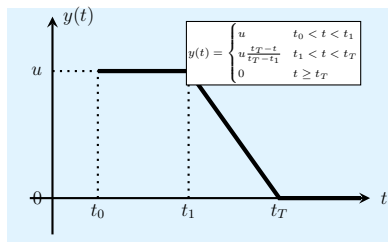
Systemem wbudowanym nazywamy dedykowany system komputerowy, który stanowi integralną część większego urządzenia lub maszyny i jest zaprojektowany do wykonywania ściśle określonego zadania [1]. W przeciwieństwie do komputerów ogólnego przeznaczenia (takich jak komputery osobiste czy smartfony), na których użytkownik może uruchamiać dowolne aplikacje, system wbudowany ma z góry zdefiniowaną funkcję i nie jest przeznaczony do późniejszych, swobodnych modyfikacji przez użytkownika końcowego.

Szczególną grupą w obrębie systemów wbudowanych są **systemy wbudowane czasu rzeczywistego**. Są to takie systemy, których poprawność wyniku przetwarzania nie zależy tylko od poprawności logicznej odpowiedzi, ale też od czasu przetwarzania, poprzedzającego odpowiedź systemu [3]. W przypadku niewypracowania odpowiedzi w zadanym czasie zdarzenie to jest traktowane jako błąd systemu, co w skrajnych przypadkach może prowadzić do awarii lub uszkodzenia fizycznego sprzętu.

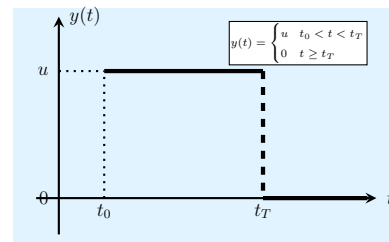
Systemy czasu rzeczywistego są powszechnie stosowane w zastosowaniach krytycznych, takich jak przemysł zbrojeniowy, aparatura medyczna czy branża motoryzacyjna, co wymaga wprowadzenia ścisłych kryteriów oceny ich niezawodności. W celu jednoznacznego zdefiniowania wymagań dotyczących determinizmu czasowego, w literaturze wprowadza się klasyfikacje systemów ze względu na rygor zachowania narzuconych terminów. W literaturze [1] systemy czasu rzeczywistego dzieli się na:

- Systemy twarde (Hard real-time systems)
- Systemy miękkie (Soft real-time systems)
- Systemy stabilne (Firm real-time systems)

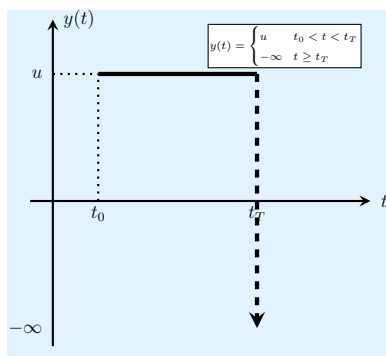
W celu lepszego zobrazowania podziału systemów czasu rzeczywistego, na rysunku rys. 2.2 przedstawiono wykresy funkcji przydatności odpowiedzi $y(t)$ od czasu jej wypracowania przez system [1].



(a) Miękki system czasu rzeczywistego



(b) Solidny system czasu rzeczywistego



(c) Twardy system czasu rzeczywistego

Rysunek 2.2: Zestawienie przebiegów funkcji przydatności odpowiedzi $y(t)$ w dziedzinie czasu [1].

Przed przystąpieniem do analizy poszczególnych wykresów, konieczne jest zdefiniowanie parametrów oraz skrótów stanowiących zmienne opisujące te funkcje:

- t – oś czasu, reprezentująca moment dostarczenia odpowiedzi przez system,
- $y(t)$ – funkcja przydatności (wartości) odpowiedzi dla obsługiwanego procesu,
- u – maksymalna, nominalna przydatność prawidłowo wypracowanego wyniku,
- t_0 – moment pojawienia się zdarzenia lub żądania wypracowania odpowiedzi przez system,
- t_1 – czas graniczny, do którego odpowiedź zachowuje pełną wartość (w systemach miękkich),
- t_T – ostateczny limit czasowy (ang. *deadline*), po przekroczeniu którego odpowiedź nie ma już pełnej wartości.

Analizując przebiegi funkcji przydatności dla poszczególnych klas systemów, można precyzyjnie scharakteryzować ich zachowanie.

Najbardziej restrykcyjne kryteria czasowe są charakterystyczne dla twardych systemy czasu rzeczywistego, co zobrazowano na rysunku 2.2c. Ich stosowanie gwarantuje bezwzględnie terminowe wypełnienie zadań krytycznych przed upływem limitu t_T . Przekroczenie czasu przetwarzania powoduje natychmiastowy spadek funkcji przydatności do wartości $-\infty$. W przypadku rozwiązań spóźniona odpowiedź jest nie tylko całkowicie bezużyteczna, ale jej pojawienie się po wyznaczonym terminie jest najczęściej równoznaczne z wystąpieniem krytycznego błędu, który może doprowadzić do poważnej awarii lub fizycznego uszkodzenia całego systemu.

Kolejną z omawianych grup stanowią miękkie systemy czasu rzeczywistego, których zachowanie zilustrowano na rysunku 2.2a. Systemy te cechują się mniejszym rygorem w kwestii terminowości, co oznacza, że zadania są wykonywane tak szybko, jak to możliwe, a ewentualne opóźnienia nie niosą za sobą skutków katastrofalnych. Jeśli odpowiedź zostanie dostarczona po czasie t_1 , ale przed ostatecznym limitem t_T , jej przydatność maleje, powodując jedynie spadek jej jakości. Przy przetwarzaniu przekraczającym czas t_T przydatność odpowiedzi wynosi zero.

Solidne systemy czasu rzeczywistego stanowią pewnego rodzaju kompromis pomiędzy twardymi, a miękkimi systemami czasu rzeczywistego. Zachowanie funkcji przydatności systemów solidnych przedstawiono na rysunku 2.2b. Każda odpowiedź dostarczona w przedziale od t_0 do t_T zachowuje stałą, pełną wartość u . W momencie przekroczenia terminu granicznego t_T przydatność wyniku natychmiastowo i skokowo spada do zera. Choć fakt spóźnienia powoduje całkowitą nieprzydatność rozwiązania, to sytuacja ta wciąż nie generuje bezpośredniego zagrożenia dla stabilności czy integralności samego systemu.

Zdefiniowane powyżej rygory czasowe określają, teoretyczny oczekiwania wobec systemu czasu rzeczywistego. W praktyce realizacja tych założeń sprowadza się do oceny zasadności stosowania **systemu operacyjnego czasu rzeczywistego** w zderzeniu z bezpośrednim programowaniem sprzętu. Decyzja ta bezpośrednio determinuje strukturę i elastyczność budowanego środowiska.

2.4. Architektura struktur jądra systemów operacyjnych

Systemy operacyjne można klasyfikować według wielu kryteriów, z których jednym z najczęściej spotykanym jest podział, ze względu na ich docelowe przeznaczenie. Przy takim podziale możemy wyróżnić systemy operacyjne generalnego przeznaczenia, systemy operacyjne rozproszone, systemy operacyjne komputerów nmainframe, systemy operacyjne serwerów oraz wiele innych [1].

Jednakże, w kontekście analizy wydajności kluczowy jest podział ze względu na strukturę systemu, która określa architekturę jego jądra. Na tym etapie jądro systemu możemy zdefiniować jedyny program, który działa w pamięci komputera nieprzerwanie przez cały czas od jego uruchomienia, pełniąc funkcję pomostu komunikacyjnego między uruchomionymi aplikacjami, a fizycznym sprzętem [2].

Konstrukcja jądra decyduje o sposobie komunikacji między usługami systemowymi, a warstwą aplikacyjną, co bezpośrednio wpływa na narzuty czasowe oraz efektywność optymalizacji kodu programów. Z uwagi na to, że przedmiotem badań w niniejszej pracy jest system Linux, analiza skupi się głównie na architekturze monolitycznej. Dla celów porównawczych zostanie ona zestawiona z inną szeroko spotykaną architekturą w systemach wbudowanych - strukturą mikrojądra.

W **architekturze monolitycznej** (ang. *monolithic kernel*) wszystkie główne zadania systemu operacyjnego są połączone w jeden rozbudowany program komputerowy [1]. Zarządzanie pamięcią, programami oraz wszystkimi sterownikami urządzeń odbywa się we wspólnej prze-

strzeni. Takie podejście sprawia, że jądro zawiera w sobie ogromną liczbę elementów, co mocno komplikuje proces jego tworzenia oraz późniejszego szukania błędów. Największą zaletą architektury monolitycznej jest jednak wysoka prędkość przetwarzania informacji – z uwagi na to, że jądro jest w całości kompilowane do jednego programu, poszczególne funkcjonalności mogą z siebie nawzajem korzystać, co znacząco przyspiesza działanie systemu. Przy architekturze monolitycznej aplikacje użytkownika są uruchamiane jako osobne procesy w chronionej pamięci. Dzięki temu jądro nie jest narażone na błędy w programach użytkownika, a różne aplikacje nie mogą nawzajem bezpośrednio mieć dostępu zasobów im nie przydzielonych. Główną wadą systemów monolitycznych pozostaje jednak słaba odporność na awarie samego jądra; jeśli błąd pojawi się w jego obrębie niego, na przykład w jednym ze sterowników, może dojść do awarii całego systemu.

Architektura mikrojądra (ang. *microkernel*) opiera się na zasadzie minimalizacji kodu wykonywanego z najwyższymi uprawnieniami procesora [2]. Kompletny system operacyjny składa się w tym przypadku z uproszczonego jądra oraz zestawu współpracujących ze sobą procesów systemowych. Samo mikrojądro realizuje jedynie fundamentalne zadania, takie jak zarządzanie wątkami oraz obsługa komunikacji międzyprocesowej [1]. Wszystkie pozostałe usługi, wliczając w to sterowniki urządzeń i systemy plików, są odseparowane od jądra i funkcjonują jako niezależne programy w przestrzeni użytkownika.

Główną zaletą takiego podejścia jest wysoka stabilność środowiska – awaria oprogramowania lub sterownika nie narusza integralności samego jądra. System potrafi zrestartować uszkodzony proces bez zaburzania pracy całego systemu. Wadą tego rozwiązania jest jednak niższa wydajność, wynikająca z narzutu czasowego na ciągłą konieczność komunikacji międzyprocesowej oraz częste przełączanie kontekstu procesora.

2.5. Studium przypadku systemu operacyjnego Linux

System operacyjny Linux wywodzi się z tradycji architektonicznej rodziny systemów "Unix", czerpiąc z niej kluczowe założenia projektowe, takie jak wielozadaniowość, wielodostępność oraz podział przestrzeni pamięci [1]. Jego rozwój rozpoczął się w 1991 roku z inicjatywy Linusa Torvaldsa, a dziś stanowi on jeden z najważniejszych i najczęściej wykorzystywanych projektów tworzonych w modelu otwartego oprogramowania (ang. *open source*) [1]. Dzięki wsparciu globalnej społeczności Linux stał się najczęściej wykorzystywanym systemem komputerowym w wielu zastosowaniach, począwszy od infrastruktury serwerowej przez systemy wbudowane, aż po chmury obliczeniowe.

Z perspektywy użytkownika klasycznym i wysoce elastycznym sposobem komunikacji z systemem Linux jest powłoka (ang. *shell*), w systemach Linux reprezentowana najczęściej przez program *bash* [1]. Z technicznego punktu widzenia jest to zwykły program działający w przestrzeni użytkownika, który pełni rolę interpretera poleceń [1]. Powłoka pozwala na uruchamianie innych programów wraz z odpowiednimi parametrami. Do jej kluczowych funkcjonalności należy elastyczne przekierowywanie standardowych strumieni wejścia i wyjścia oraz łączenie pojedynczych programów w potoki (ang. *pipelines*), co pozwala na przekazywanie wyników między różnymi procesami [1]. Ponadto powłoka umożliwia asynchroniczne uruchamianie zadań w tle oraz tworzenie skryptów, które dzięki obsłudze zmiennych i instrukcji sterujących (np. pętli czy instrukcji warunkowych) pozwalają na automatyzację złożonych operacji systemowych [2].

Kluczowym elementem systemu jest jądro o strukturze monolitycznej [4], które odpowiada za bezpośrednią kontrolę nad najważniejszymi zasobami systemu. Z uwagi na tę architekturę oraz konieczność zapewnienia bezpieczeństwa, aplikacje działające w przestrzeni użytkownika

nie mają bezpośredniego dostępu do krytycznych zasobów systemu. Wszelka interakcja z zasobami zarządzanymi przez system musi odbywać się za pośrednictwem zdefiniowanego interfejsu wywołań systemowych, który w systemie Linux jest zgodny ze standardem **POSIX** [1]. Każde takie wywołanie wymusza sprzętowe przełączenie procesora do trybu jądra. Operacja ta wymaga zapisania stanu rejestrów oraz zmiany kontekstu wykonania, co wiąże się z dodatkowym narzutem czasowym.

2.5.1. Procesy, wątki w systemie Linux

W przeciwieństwie do wielu innych systemów operacyjnych, Linux nie wprowadza ścisłego, wewnętrznego rozróżnienia między procesami, a wątkami [1]. Każdy kontekst wykonania, niezależnie od tego, czy jest to jednowątkowy proces, czy jeden z wielu wątków wewnątrz procesu, jest reprezentowany w jądrze jako zadanie (ang. *task*) za pomocą struktury danych *task_struct* [4].

Klasyczne tworzenie nowego procesu odbywa się poprzez wywołanie systemowe *fork*. Aby uniknąć kosztownego i nieefektywnego kopiowania całej pamięci, Linux stosuje optymalizację kopiowania przy zapisie (ang. *copy on write*) [4]. Początkowo proces potomny otrzymuje własne tablice stron, jednak wskazują one na fizyczne strony pamięci procesu macierzystego z atrybutem tylko do odczytu. Nowa kopia strony jest alokowana w pamięci RAM dopiero w momencie, gdy któryś z procesów spróbuje dokonać na niej operacji zapisu [1].

Z kolei fundamentalną różnicą w architekturze Linuksa, w porównaniu do klasycznych systemów UNIX, jest obsługa wątków. Wprowadzono w nim wywołanie systemowe *clone*, które zaciera tradycyjną różnicę między procesami a wątkami [1], umożliwiając tworzenie nowego wątku z bardzo podobnymi cechami do klasycznego procesu.

Pozwala ono zminimalizować narzut systemowy poprzez precyzyjne sterowanie tym, co jest kopiowane, a co współdzielone. Wywołanie to przyjmuje jako argument mapę bitową, w której poszczególne flagi (np. *CLONE_FILES*, *CLONE_SIGHAND*) decydują o współdzieleniu deskryptorów plików czy tablicy obsługi sygnałów [1].

2.5.2. Szeregowanie i synchronizacja w systemie Linux

System operacyjny Linux posiada przemyślany i wysoce elastyczny mechanizm szeregowania zadań oraz synchronizacji, który obsługuje zarówno standardowe programy użytkowe, jak i zadania czasu rzeczywistego [1].

W kontekście planowania czasu procesora, jądro wyróżnia trzy główne klasy wątków: zadania czasu rzeczywistego typu FIFO, zadania czasu rzeczywistego z rotacją (ang. *round-robin*) oraz zadania standardowe [1]. Wątki czasu rzeczywistego otrzymują zawsze najwyższe priorytety (w skali od 0 do 99). Mimo swojej nazwy nie dają one ścisłych gwarancji dotrzymania terminów (ang. *deadlines*), a jedynie zapewniają bezwzględne pierwszeństwo wykonania przed zwykłymi aplikacjami, [1]. Jest to bardzo istotna cecha systemu Linux, która pokazuje, że nie jest on odpowiedni (w klasycznej, nieprzerobionej wersji systemu) do bezpośredniego wykorzystania w twardej systemach czasu rzeczywistego [4]. Wątki typu FIFO działają aż do momentu dobrowolnego oddania procesora lub wyłączenia przez zadanie o wyższym priorytecie, podczas gdy wątkom szeregowanym cyklicznie przydzielane są dodatkowo określone kwanty czasu [1].

Z kolei standardowe zadania o niższych priorytetach (od 100 do 139) są z reguły obsługiwane przez algorytm *Completely Fair Scheduler* (CFS) [1], który optymalizuje i sprawiedliwie rozdziela czas procesora z wykorzystaniem struktury drzewa czerwono-czarnego [2].

Uzupełnieniem efektywnego zarządzania zadaniami są zróżnicowane mechanizmy syn-

chronizacji, które zastąpiły historyczną, nieefektywną globalną blokadę jądra. Aby zminimalizować ryzyko konfliktów o dostęp do zasobów, Linux udostępnia pełen przekrój rozwiązań: od operacji atomowych i barier pamięci, poprzez blokady wirujące (ang. *spinlocks*) [4] – optymalne, gdy wątek nie powinien być usypiany – aż po wysokopoziomowe muteksy i semaforey dla zadań mogących bezpiecznie przejść w stan oczekiwania [1].

2.5.3. Zarządzanie pamięcią

W systemie Linux pamięć jest stronicowana. W związku z tym, podstawową jednostką do zarządzania pamięcią fizyczną jest struktura o nazwie *page* [4]. Z uwagi na ograniczenia sprzętowe niektórych architektur, pamięć fizyczna została logicznie podzielona na strefy (ang. *zones*). Należą do nich między innymi *ZONE_DMA*, przeznaczona na operacje bezpośredniego dostępu do pamięci, *ZONE_NORMAL*, zawierająca standardowe strony, oraz *ZONE_HIGHMEM* [1].

Każdy proces uruchomiony w systemie operuje na własnej wirtualnej przestrzeni adresowej, która logicznie dzieli się na trzy główne segmenty: kod, dane oraz stos [1]. Segment kodu przechowuje instrukcje programu i domyślnie posiada atrybut umożliwiający jedynie jego odczyt, co pozwala na bezpieczne współdzielenie go między wieloma jednoczesnymi uruchomieniami tego samego programu [2]. Segment danych przechowuje zmienne globalne i statyczne, przy czym dla zmiennych niezainicjowanych Linux stosuje optymalizację: mapuje je na jedną fizyczną, wyzerowaną stronę, dopóki proces nie spróbuje dokonać modyfikacji, co wyzwala mechanizm kopiowania przy zapisie (ang. *copy-on-write*) [1]. Ponadto jądro oferuje mechanizm mapowania plików w pamięci, co umożliwia procesom bezpośredni dostęp do danych plikowych lub współdzielonych bibliotek z pominięciem standardowych wywołań systemowych [1].

Do zarządzania przydziałem fizycznych stron pamięci Linux wykorzystuje **algorytm bliźniaków** (ang. *buddy algorithm*) [1]. Choć mechanizm ten jest w większości przypadków wysoce wydajny, może prowadzić do zjawiska fragmentacji wewnętrznej w przypadku konieczności alokacji struktur o rozmiarach znacznie mniejszych niż domyślne wielkości strony. Aby zminimalizować ten problem i zredukować narzut systemowy, system wdraża mechanizm dyspozytora płytowego (ang. *slab allocator*) [4], który działa dysponuje pamięcią poprzez algorytm bliźniaków, aby później podzielił stronę na mniejsze fragmenty i umożliwić jej zarządzanie [1].

Na poziomie interfejsu programistycznego jądra alokacja pamięci odbywa się za pomocą kilku wyspecjalizowanych funkcji do zarządzania pamięcią. Warto tutaj wspomnieć o tych najpopularniejszych, czyli *kmalloc*, gwarantująca przydział obszaru ciągłego zarówno w przestrzeni wirtualnej, jak i fizycznej [4] oraz *vmalloc* wykorzystywana do alokacji większych struktur, niewymagających ciągłości fizycznej. Ta druga jednak wiąże się z większym narzutem wydajnościowym wynikającym z konieczności modyfikacji tablic stron [4].

3. WYBRANY ALGORYTM PRZETWARZANIA DANYCH I TECHNIKI OPTYMALIZACJI PROGRAMÓW KOMPUTEROWYCH

3.1. *Opis i analiza Szybkiej Transformaty Fouriera*

3.2. *Warianty algorytmu Szybkiej Transformaty Fouriera*

3.3. *Opis procesu kompilacji programu komputerowego*

3.4. *Przegląd najpopularniejszych kompilatorów języka C*

3.5. *Poziomy optymalizacji dostępne w kompilatorach Clang i Gcc*

3.6. *Zaawansowane techniki optymalizacji kodu*

4. PROJEKT SYSTEMU TESTOWEGO I METODYKA PRZEPROWADZONYCH BADAŃ

4.1. Założenia projektowe i architektura aplikacji

4.2. Opis platformy sprzętowej

4.3. Użyte narzędzia programistyczne

4.4. Metodyka przeprowadzonych pomiarów

4.5. Prezentacja przygotowywanych wariantów programu

4.6. Implementacja wersji wielowątkowej, opis biblioteki pthread

4.7. Weryfikacja poprawności obliczeń z biblioteka zewnętrzna

5. ANALIZA WYNIKÓW PRZEPROWADZONYCH BADAN

5.1. Analiza wpływu flag kompilatora na kod sekwencyjny

5.2. Analiza wpływu optymalizacji ręcznej programu sekwencyjnego

5.3. Analiza skalowalności rozwiązania wielowątkowego

5.4. Zestawienie zbiorcze i dyskusja wyników

6. PODSUMOWANIE I WNIOSKI

6.1. *Wnioski koncowe*

6.2. *kierunek dalszych badan*

SPIS RYSUNKÓW

2.1	Warstwowy model systemu komputerowego	10
2.2	Zestawienie przebiegów funkcji przydatności odpowiedzi $y(t)$ w dziedzinie czasu [1].	14

SPIS TABEL

BIBLIOGRAFIA

- [1] Andrew S. Tanenbaum and Herbert Bos. *Systemy operacyjne*. Helion, Gliwice, 2024. Przekład: Radosław Meryk.
- [2] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. J. Wiley & Sons, Hoboken, N.J., 8th ed. edition, 2008.
- [3] K. C Wang. *Embedded and Real-Time Operating Systems*. Springer Nature, Cham, 1 edition, 2017.
- [4] Robert Love. *Linux kernel development*. Developer's library : essential references for programming professionals Linux kernel development. Addison Wesley, Place of publication not identified, 3rd ed. edition, 2010.